

Programação Orientada a Objeto

UNIVERSIDADE DO VALE DO SAPUCAÍ – UNIVAS – POUSO ALEGRE/MG
PROFESSOR: RAFFAEL CARVALHO

Relacionamentos e Composição de Objetos

Conceitos já estudados

O que são Classes e Objetos: A diferença entre o molde (Paciente.java) e a instância (new Paciente()).

Encapsulamento: A importância de proteger atributos com private e expor o acesso via Getters e Setters.

Organização: Como estruturar nosso código em pacotes (modelo, visao, controle) seguindo o padrão MVC

Como conectar esses objetos de forma inteligente e profissional?

Conectando os Dados

Vamos analisar nosso sistema de Clínica Médica. Um Agendamento não existe sozinho. Ele fundamentalmente depende de outras duas entidades:

- Um **Paciente** que será atendido.
- Um **Medico** que realizará o atendimento.

Qual a melhor forma de representar essa conexão no código?

A Conexão por "RG" (Usando IDs)

Uma primeira ideia seria fazer o Agendamento guardar apenas a "identidade" do paciente e do médico.

```
1  // Abordagem inicial e INEFICIENTE
2  public class Agendamento {
3      private int idAgendamento;
4      private int idPaciente; // <-- Guarda só o número
5      private int idMedico;   // <-- Guarda só o número
6      // ...
7  }
```

As Limitações da Abordagem por IDs

Trabalhar com IDs quebra princípios importantes da POO:

Baixa Coesão: O objeto Agendamento não é autossuficiente. Para saber o nome do paciente, ele depende de um código externo para buscar essa informação.

Alto Acoplamento: A classe que manipula o agendamento (uma Tela no pacote visão, por exemplo) fica com a responsabilidade de buscar o Paciente e o Medico em listas separadas. Isso mistura responsabilidades.

Código Verboso e Repetitivo: Toda vez que precisarmos do nome, telefone ou especialidade, teremos que escrever um loop de busca.

```
1  Agendamento ag = ...; // um agendamento qualquer
2  Paciente p = buscarPacientePorId(listaDePacientes, ag.getIdPaciente()); // Trabalho extra!
3  Medico m = buscarMedicoPorId(listaDeMedicos, ag.getIdMedico()); // Trabalho extra!
4
5  System.out.println("Consulta de " + p.getNome() + " com Dr(a). " + m.getNome());
```

A Relação de Composição ("tem-um")

A Programação Orientada a Objetos nos oferece uma solução mais elegante: a Composição.

A ideia é simples: se um **Agendamento** tem um **Paciente**, então a classe **Agendamento** deve conter o objeto **Paciente** inteiro, e não apenas seu ID.

Agendamento tem-um **Paciente**.

Agendamento tem-um **Medico**.

Refatorando o Agendamento para Composição

```
1  // Abordagem com COMPOSIÇÃO - CORRETA E ROBUSTA
2  public class Agendamento {
3      private int idAgendamento;
4      private Paciente paciente; // <-- Objeto Paciente completo!
5      private Medico medico;     // <-- Objeto Medico completo!
6      private LocalDate data;
7      private LocalTime hora;
8
9      // Construtor garante que o Agendamento já nasce completo
10     public Agendamento(int id, Paciente p, Medico m, ...) {
11         this.paciente = p;
12         this.medico = m;
13         //...
14     }
15 }
```

Código Limpo e Intuitivo

Com a nova classe Agendamento, o código na Tela (Visão) fica assim:

```
1  // CÓDIGO EM UMA TELA (VISA0)
2  Agendamento ag = ...; // o mesmo agendamento
3
4  // Acesso direto, limpo e orientado a objetos!
5  System.out.println(
6      "Consulta de " + ag.getPaciente().getNome() +
7      " com Dr(a). " + ag.getMedico().getNome()
8  );
```

O código se torna mais legível e a **responsabilidade** de "conhecer" o paciente e o médico é do próprio Agendamento, como deve ser.

Impacto na Persistência (ManipuladorArquivos)

No arquivo .csv nós continuamos salvando os IDs, certo?

Exato! A composição é um conceito da modelagem de objetos em memória. A persistência é um problema separado.

A classe ManipuladorArquivos atua como uma "ponte":

- Ao ler do CSV:
 1. Lê as linhas e os IDs.
 2. Busca os objetos Paciente e Medico correspondentes (em listas já carregadas).
 3. Usa os objetos encontrados para instanciar Agendamento com o construtor parametrizado.
- Ao salvar no CSV:
 1. Recebe um objeto Agendamento.
 2. Pega o objeto Paciente de dentro dele e chama getPaciente().getId().
 3. Pega o objeto Medico e chama getMedico().getId().
 4. Escreve os IDs no arquivo.

Exercício

- Estude o código do projeto “clínica” fornecido, em seguida implemente as novas funcionalidades aprendidas nessa aula no projeto “biblioteca”. Provavelmente você vai precisar alterar a classe “empréstimo” como foi alterada a classe “agendamento” nessa aula.

Bibliografia Básica

- HARBOUR, Jonathan S. Programação de games com JAVA: Tradução da 2ª edição norte-americana. Cengage Learning Brasil, 2009. E-book. ISBN 9788522127290. Disponível em: <https://integrada.minhabiblioteca.com.br/#/books/9788522127290/>. Acesso em: 26 set. 2023. (MINHA BIBLIOTECA)
- HORSTMANN, Cay; CORNELL, Gary. Core Java: Volume I - Fundamentos. São Paulo: Pearson, 2010.
- SIERRA, Kathy; BATES, Bert. Use a Cabeça Java. 2.ed. São Paulo: Alta Books, 2010
- FORBELLONE, A. L. V.; EBERSPACHER, H. F. Lógica de programação. São Paulo: Makron Books, 2000.